

LPDB: Lightweight Policy-Driven Blockchain With Batch Verification for Rail Transit Systems

Mengjie Zhou , Xiaotong Zhou , Wei Wu , and Debiao He , *Member, IEEE*

Abstract—Rail transit is a commonly used mode of transportation for daily commuting, but traditional Rail Transit Systems (RTS) typically face significant challenges such as high mobility and low latency. Furthermore, the RTS generates a substantial volume of real-time interactive data, posing difficulties in ensuring user privacy while maintaining practicality. To address these concerns, blockchain as a promising technology, has been leveraged to safeguard data transmission and access. However, public blockchains such as Bitcoin and Ethereum may not be suitable for real-time interactive RTS scenarios due to their lack of flexibility and control. To overcome this, the notion of the Policy-Driven Blockchain (PDB) was introduced, which is a private blockchain with flexible policy control and management mechanisms. PDB can be quickly customized and deployed according to specific business requirements and rules, making it a valuable feature for RTS scenarios. However, their scheme relies on bilinear pairings, which incurs high computational overhead and does not support batch validation. In this article, we propose a Lightweight Policy-Driven Blockchain (LPDB) scheme to overcome these challenges, the core of which is the design of a Pairing-free Policy-Driven Signature (PFPDS) scheme better tailored to meet the specific requirements in RTS. In addition, considering the large number of communication messages in RTS, we analyze the advantages of batch verification of signatures. In conclusion, the security of our proposal is validated under the random oracle model, and we evaluate the computational expenses associated with the algorithms via simulations on a laptop. The implementation outcomes demonstrate the practicality and viability of our scheme.

Index Terms—Batch verification, blockchain, no bilinear pairs, rail transit.

I. INTRODUCTION

THE advancement of the Internet of Things (IoT), vehicle technology, satellite communication [1], [2], and Cyber-Physical Systems (CPS) has led to trains becoming more intelligent and autonomous than they have ever been before [3], [4], [5]. The concept of Intelligent Transportation Systems (ITS) [6] has emerged to enhance traffic network security, minimize traffic congestion, and improve user experience [7].

The Rail Transit System (RTS) is an important branch of the ITS, which aims to improve transportation efficiency, security, and reliability [8]. As shown in Fig. 1, the RTS is composed of different components, which consist of the Network Control System (NCS), On-Board System (OBS), Trackside System (TS), and Communication System (CS) [9]. The Train Control System (TCS) and Authorization Management System (AMS) are incorporated in the RTS and are highly advanced. The TCS is managed by the train dispatcher, who assumes responsibility for all train operation-related duties within the station. The dispatcher's request is sent to the authorization management system, which checks the request message, and then sends a command to the OBS to execute the rule. The OBS is responsible for determining the train's position and speed. The TS reports the train's status to the AMS regularly via the CS. To realize the timely updating and sharing of train status information, which brings great convenience to people's transportation [9].

RTS is one of the most popular passenger and freight transportation systems [10], which not only brings convenience to drivers, it can also monitor traffic flow in real-time, make statistics on the running conditions of trains, and help trains better cope with some emergencies on the tracks. At the same time, it also faced some problems. For example, A growing number of sensor devices are producing and sharing vast quantities of data, which will result in enormous network traffic being managed [11]. Simultaneously, the RTS will encounter substantial obstacles in achieving high mobility, low latency, and effective data storage and management. Moreover, ensuring user privacy and achieving anonymity remains a challenge. In addition, The traditional RTS is completely centralized and mainly depends on the AMS. The practicability is very limited and only users who are registered by the system and become train dispatchers can access the data to verify the train speed, location, etc.

To address the challenges faced by RTS, various technologies can be utilized, one of which is blockchain. Blockchain has several characteristics that make it a suitable technology for use in RTS applications. For instance, it is decentralized, open, secure, and anonymous [7]. However, traditional public

Manuscript received 26 May 2023; revised 18 September 2023 and 21 October 2023; accepted 4 November 2023. Date of publication 8 November 2023; date of current version 22 April 2024. This work was supported in part by the National Natural Science Foundation of China under Grants U21A20466, 61972294, 61932016, and 62372108, in part by the Science Foundation of Fujian Provincial Science and Technology Agency under Grant 2020J02016, in part by the New 20 Project of Higher Education of Jinan, China under Grant 202228017, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20220935, and in part by the Fundamental Research Funds for the Central Universities under Grant 30922010917. The review of this article was coordinated by Prof. Luca D'Acerno. (Corresponding author: Xiaotong Zhou.)

Mengjie Zhou is with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China (e-mail: zhoumengjie00@163.com).

Xiaotong Zhou is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: xtzhou163@163.com).

Wei Wu is with the School of Mathematics and Statistics, Fujian Normal University, Fuzhou 350117, China (e-mail: weiwu@fjnu.edu.cn).

Debiao He is with the Shandong Provincial Key Laboratory of Computer Networks, Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China, and also with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: hedebliao@163.com).

Digital Object Identifier 10.1109/TVT.2023.3330889

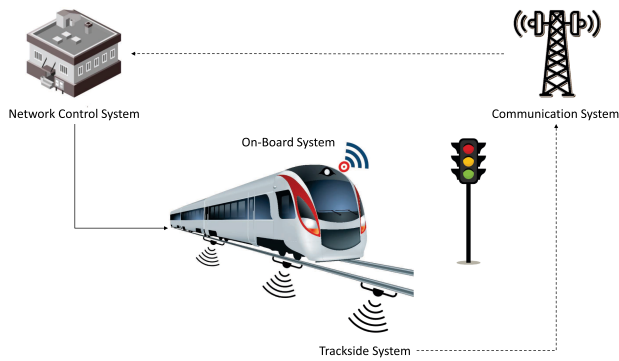


Fig. 1. Traditional rail transit system.

blockchains like Bitcoin and Ethereum [12] may not be suitable for all RTS scenarios, as they may not provide the necessary flexibility and control required by certain applications. This limitation becomes especially apparent in the context of RTS. Therefore, private or consortium blockchains are better suited to fulfill this crucial role. These types of blockchains provide more control and flexibility and can be tailored to meet the specific needs of an RTS application.

Building upon this foundation, Mu et al. [9] explore the concept of the Policy-Driven Blockchain (PDB), which is a private blockchain that features flexible policy control and management mechanisms. Policy sets are utilized for access control, where users are only granted access to resources if they meet the policy set requirements, thereby limiting participation in specific transactions to signatories who satisfy the policy set criteria. Consequently, PDB can be quickly customized and deployed based on different business needs and rules. We are trying to use PDB to solve the problems faced in RTS, whereas, currently, the core scheme Policy-Driven Signature (PDS) in this type of blockchain incurs high computational costs and may not meet the lightweight computing requirements in RTS scenarios. Wherein, the main reason for the insufficient practicality of existing PDB is that the efficiency of the PDS involved in this technology can be further improved. Firstly, it is based on bilinear pairings, which incurs high computational costs. Secondly, the signature scheme is difficult to support batch verification. It is challenging to satisfy the high-efficiency batch verification requirements of blockchain due to its typically large transaction volume.

To differentiate our scheme from this, we proposed a Lightweight Policy-Driven Blockchain (LPDB), which primarily focuses on devising an appropriate signature scheme that satisfies the unique demands of the RTS, called the Pairing-free Policy-Driven Signature (PFPDS) scheme. Our PFPDS scheme exhibits the following favorable characteristics:

- 1) Our PFPDS scheme avoids the high-cost bilinear pairing operation, performs off-chain signature verification, and only stores the results of verification in the blockchain. Moreover, our scheme can also facilitate batch verification of signatures.
- 2) Based on the Elliptic Curve Discrete Logarithm Problem (ECDLP) assumption, our signature scheme has been

demonstrated to be secure against two types of adversarial attacks under the Random Oracle Model (ROM). As demonstrated by experiments, it can effectively improve computational efficiency and reduce communication complexity.

- 3) We have constructed LPDB and applied it to RTS, where our focus was solely on demonstrating the legitimacy and completeness of transactions. We did not take into account the development of underlying consensus mechanisms.

A. Related Work

Currently, most of the research on RTS is focused on energy-saving solutions for collaborative control of trains, optimizing routes, or enhancing the user experience. However, for a secure RTS, the privacy and security of data information involved in this system must be taken into account.

Attribute-Based Signatures (ABS) are digital signature technologies that enable a signer to select attributes that meet authentication policies to generate valid signatures without disclosing user identities or attributes. Maji et al. [13] first proposed it in 2008. Then, Shahandashti and Safavi Naini provided a signature design based on threshold attributes [14]. The author's proposed methodology is centered around demonstrating the ownership of a signature by disclosing only the necessary attributes, thereby preserving the confidentiality of each signer's attributes. The proposal by Huang et al. [15] involved an attribute-based signature scheme that was utilized in vehicular ad hoc networks (VANETs). However, the challenge for this scheme is the anonymity in VANETs and how to achieve more efficient authentication.

With the recent application of blockchain technology in computing environments, the multi-authority attribute-based signature (MA-ABS) scheme proposed by Guo et al. [16] in 2018 aimed to achieve anonymous authentication of electronic health records (EHRs) within blockchains. Sun et al. [17] suggested a decentralized ABS for medical blockchains and established a secure storage and verification system that employs both on-chain and off-chain collaborative storage models to enable the secure share of EHR data among various attribute permissions. In addition, in Wang et al.'s scheme [18], ABS and IBS have found applications in healthcare blockchains as well. Recently, more and more progress has been made in the study of ABS solutions for resource-constrained devices [19], [20], [21], [22], [23], [24], [25], [26]. In a separate study by Lin et al. [27], a blockchain system was employed for mutual authentication, wherein an ABS scheme was utilized to achieve anonymous authentication. Datta et al. [28] proposed an ABS that features adaptive unforgeability and complete privacy for signers.

In contrast, traditional ABS schemes need a large number of parameter sets, so the key and signature are usually long. Meanwhile, as Key Generation Center (KGC) generates private signature keys for users, users can be fully controlled. Al-Riami et al. [29] first introduced the concept of certificateless signatures in 2003, aiming to address the key escrow issue. That is, a certificateless signature does not necessitate a completely reliable KGC. Combined with the above, Mu et al. [9] proposed

a PDS algorithm, in which the author grants users access to resources by setting a policy set, and the user's authority is controlled by a specific policy set. However, the challenge they face is that involving bilinear pairs, it is difficult to fit lightweight computing requirements into RTS scenarios, at the same time, their proposed scheme did not realize batch verification.

In the Internet of Vehicles (IoV), where current schemes can support batch verification [30], [31], [32], [33], typically, the Authority generates a pseudonym for the user/vehicle to preserve the anonymity of the user/vehicle and safeguard their privacy. Nevertheless, the generation of pseudonyms often relies on a trusted third-party and can be a complex procedure.

With the development of ITS, the technological advancements involved in RTS have become an urgent issue that needs to be addressed. However, most of the aforementioned research has been applied to the IoV or VANETs. Therefore, our research plays a positive role in the development of ITS. Additionally, RTS typically faces stringent resource limitations in terms of computation, storage, and communication. To adapt to these constraints, we have proposed a new scheme that aims to reduce the system's resource requirements.

B. Organization

The subsequent sections of this paper are structured as follows. Section II provides a summary of the symbol notation and cryptographic primitives employed in the proposed scheme. In Section III, we introduce our system definition, which includes system architecture, adversarial model, and scheme definition. Section IV outlines the implementation of the proposed scheme within the transportation system, encompassing the PFPDS and its accompanying correctness analysis. In Section V, we provide a security proof and probability analysis of the scheme under the ROM. To validate the feasibility of our scheme in practical applications, Sections VI and VII present that the specific application and performance analysis of the scheme in RTS, respectively. To summarize, we conclude this paper in Section VIII.

II. PRELIMINARY KNOWLEDGE

In this section, we will describe the involved symbols, Schnorr signature scheme, and the ECDLP assumptions are as follows.

A. Involved Symbols

Table I illustrates the primary symbols utilized in our scheme.

B. Schnorr Signature Scheme

In cryptography, a Schnorr signature is a digital signature scheme described by C. P. Schnorr [34], whose security is based on the ECDLP [35]. It mainly includes three algorithms:

Key Generation: Choose an elliptic curve $E: y^2 = x^3 + ax + b \pmod{p}$, where $4a^3 + 27b^2 \neq 0$, and a base point G . Select d_A randomly as the user's secret key ($d_A < n$, n is the order of G). Calculate $Q_A = d_A \cdot G$ as the user's public key.

Signing: Denote m as a message chosen from the message space $\mathbb{M}$ that is to be signed. The following steps are required to generate the signature on m . To begin with, select a random

TABLE I
SYMBOL DESCRIPTION

Notation	Description
p	Large prime number
$\mathbb{G}$	Elliptic curve
G	Generator of elliptic curve
Pol_i	Policy Set
$params$	System public parameters
$H(\cdot)$	Cryptographic hash functions
x_i	Secret value selected by user
s_i	Partial private key generated by KGC
sk_i	User's private key
pk_i	User's public key
sk_{KGC}	System master key
pk_{KGC}	KGC's public key
m_i	Message
σ_i	Signature on message m_i
$\mathcal{C}$	Challenger
$\mathcal{A}_1/\mathcal{A}_2$	Two types of adversaries
CDHP	Computational Diffie-Hellman Problem
BSDHP	Bilinear Strong Diffie-Hellman Problem

number k and calculate $R = k \cdot G$, where $k < n$; after that, calculate $S = k + \text{hash}(m||R) \cdot d_A \pmod{n}$; whereupon, the signature of message m is returned as (R, S) .

Verification: To determine whether (R, S) constitutes the signature of message m , this algorithm verifies the equation $S \cdot G = \text{hash}(m||R) \cdot Q_A + R$, and if the equation holds, the signature is deemed valid and accepted.

C. ECDLP

In 1987, Koblitz [36] constructed ECDLP using Abelian additive groups formed by points on elliptic curves [37], [38]. Given the prime number p and elliptic curve $\mathbb{G}$, where G, P belong to $\mathbb{G}$, the adversary's goal is to find the positive integer $s \in \mathbb{Z}_p^*$ such that $P = s \cdot G$.

Definition 1 (ECDLP assumption): If there is no polynomial time t algorithm with a non-negligible ξ advantage to solve ECDLP in $\mathbb{G}$, then (Q, t, ξ) -ECDLP assumption holds in $\mathbb{G}$.

III. SYSTEM DEFINITION

In this section, a concise introduction is presented regarding the system model, scheme definition, and countermeasure model, which pave the way for the following core work.

A. System Architecture

Our scheme, depicted in Fig. 2, relies on a novel LPDB, whose system structure is chiefly composed of three types of entities, namely, System Administrator (SA), Blockchain (BC), and BCUser. BCUsers are divided into several groups according to their respective policies, where BCUsers include trains, drivers, sensors, etc. Despite sharing the same set of policies, BCUsers within each group possess distinct private keys.

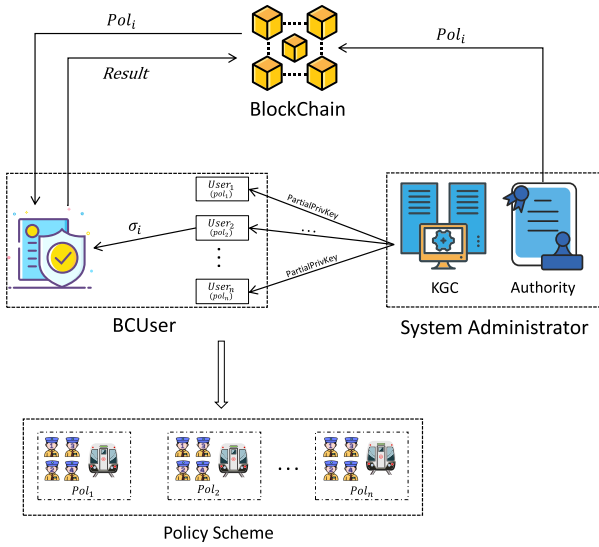


Fig. 2. Architecture diagram of system model and policy scheme.

System Administrator: The SA includes the Authority and KGC to maintain the entire RTS. The SA is considered completely trustworthy with no opponent to compromise with and has the following features:

- 1) The Authority is tasked with defining regulations, initializing the system, and creating policy sets where the policy set limits the power of the user.
- 2) For each user in the system, the KGC generates partial private keys (*PartialPrivKey*) via the master key sk_{KGC} and the corresponding policy. It's worth noting that our system maintains the confidentiality of user signature keys, preventing their disclosure to the system administrator.

Blockchain: It is a decentralized and disintermediated technology with immutable, open, and transparent information. The database is maintained collectively and is reliable. In our paper, BC is designated to store both the policy set Pol_i and the results of signature verification. The user can read the policy set information and the signature verification result from the BC to authenticate the signature's validity.

BCUser: Blockchain users are divided into several groups based on different policies, including trains, drivers, sensors, and others. Among them, sensors are an important component of BCUsers. This role indicates a BC user and implements the following functions:

- Receive partial private key *PartialPrivKey* from KGC.
- By utilizing the user's randomly selected secret value in combination with the partial key, the private key, sk_i , and its corresponding public key, pk_i are generated.
- The main role of the Sensor is to collect traffic information and provide signature verification. Transactions are executed by generating a PFPDS by utilizing the sk_i of the user and verifying it off-chain.

Policy Scheme: Regarding the grouping method for BCUser, there is a policy-driven scheme implemented in our LPDB.

Now, let us provide a brief description of the policy scheme. Firstly, the SA assigns a policy $(pol_1, pol_2, \dots, pol_n)$ to each BCUser as their unique identifier. Secondly, the SA classifies BCUsers based on *rules*, meaning that users belonging to the same rules will be grouped together in the same policy set Pol_i . Subsequently, users will communicate using their respective policy sets.

B. Formal Definition of PFPDS

The framework of PFPDS is defined as follows.

- $(pol_1, pol_2, \dots, pol_n) \leftarrow \text{PolicyGen}$: This algorithm produces a set of policy attributes $(pol_1, pol_2, \dots, pol_n) \in \{0, 1\}^*$ based on the *rules* established by the Authority.
- $(params) \leftarrow \text{Setup}$: The KGC produces the system public parameters, $params$, and master private key, sk_{KGC} , by utilizing a security parameter, λ .
- $(x_i) \leftarrow \text{SetSecretValue}$: After receiving the $\{params, Pol_i\}$, the signer chooses and outputs the secret value, x_i .
- $(s_i, A_i) \leftarrow \text{SetPartialPrivKey}$: On input $\{params, sk_{KGC}\}$, and user's policy sets $Pol_i = pol_j || \dots || pol_k$ for $\{pol_j, \dots, pol_k\} \in \{0, 1\}^*$, where $||$ denotes a bitwise concatenation. KGC outputs A_i, B_i , and partial private key s_i , and checks if s_i is valid.
- $(pk_i, sk_i) \leftarrow \text{SetUserKey}$: By inputting $\{params, Pol_i\}$, the signer generates a public-private key pair, (pk_i, sk_i) .
- $(\sigma_i) \leftarrow \text{Sign}$: The signer outputs the signature σ_i , by utilizing the message $\{params, sk_i, m_i\}$, where $m_i \in \mathbb{Z}_p^*$.
- $(0/1) \leftarrow \text{Verify}$: After receiving the message $\{params, \sigma_i, m_i, pol_i || \dots || pol_k, pk_i\}$, the algorithm verifies the signature's validity. For a valid signature, the output of the algorithm is 1; otherwise, it is 0.
- $(0/1) \leftarrow \text{BatchVerify}$: Given public parameters $params$, policy set Pol_i , public key pk_i , mass messages m_i ($i = 1, 2, \dots, n$), and their corresponding signatures σ_i ($i = 1, 2, \dots, n$). If the signatures are successfully verified in batches, the algorithm outputs 1; otherwise outputs 0.

C. Adversarial Model

Our construction considers two distinct types of adversaries, which are defined as follows:

- 1) Type I adversary ($\mathcal{A}_1$): The first type of attacker can either query the user's public key or substitute it with a fraudulent one, but lacks knowledge of the system's master key; *Malicious User*.
- 2) Type II adversary ($\mathcal{A}_2$): The second type of attacker can acquire the system's master key but is unable to substitute the legitimate user's public key or make queries about the user's public key; *Malicious KGC*.

The existential unforgeability of the PFPDS scheme is delineated by the depiction of two games, *Game-I* and *Game-II*, as follows:

Game-I: In this game, the challenger $\mathcal{C}$ and the adversary $\mathcal{A}_1$ engage in an interaction.

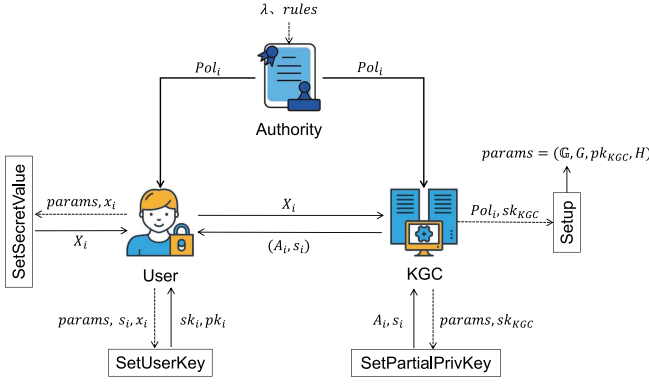


Fig. 3. Process of setup and generating User's public-private key pairs.

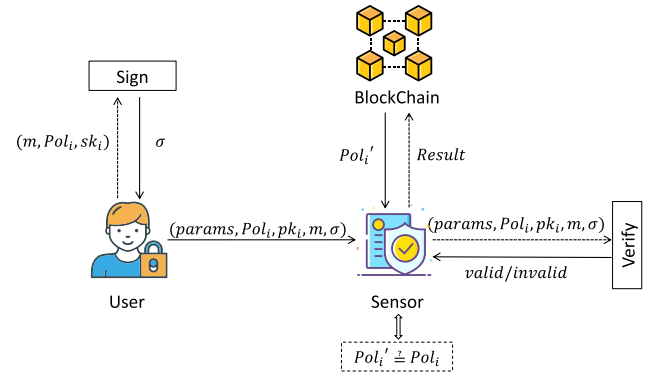


Fig. 4. Signing and verifying stage.

- **Initialization:** At this stage, $\mathcal{C}$ executes the *setup* algorithm to generate the system public parameters, pms , the system master key, msk , and the master public key, mpk . Subsequently, $\mathcal{C}$ transmits the pms and mpk to $\mathcal{A}_1$, while maintaining msk secret.
- **Queries:** At this time, $\mathcal{A}_1$ asks the following oracle:
SetPartialPrivKey: The user's partial private key, psk , is generated by $\mathcal{C}$ and then send to $\mathcal{A}_1$.
SetUserKey: $\mathcal{C}$ sends the user's public-private key pair (pk, sk) to $\mathcal{A}_1$.
PubKeyReplace: $\mathcal{A}_1$ sets pk^* as the user's new public key and sends it to $\mathcal{C}$.
Sign: $\mathcal{A}_1$ wants to query signature on (Pol_i, m_i) , $\mathcal{C}$ sends a signature σ_i to $\mathcal{A}_1$.
- **Forgery:** $\mathcal{A}_1$ outputs a message-signature pair, (m_i^*, σ_i^*) , over the policy set, Pol_i^* , and subject to the following conditions:
 - $\mathcal{A}_1$ has made no queries regarding the *SetPartialPrivKey* and *Sign* algorithm under (Pol_i^*, pk^*, m_i^*) .
 - The signature σ_i^* generated by $\mathcal{A}_1$ can be verified, where pk^* is selected by $\mathcal{A}_1$.

Game-II: In this game, the challenger $\mathcal{C}$ and the adversary $\mathcal{A}_2$ engage in an interaction.

- **Initialization:** This stage is similar to *Game-I*. However, $\mathcal{C}$ also needs to send the msk to $\mathcal{A}_2$ at the same time.
- **Queries:** The oracle query is similar to *Game-I*.
- **Forgery:** $\mathcal{A}_2$ outputs a message-signature pair, (m_i^*, σ_i^*) , on the policy set, Pol_i^* , and subject to the following conditions:
 - $\mathcal{A}_2$ has not inquired about the *SetSecretValue* and *Sign* algorithm under (Pol_i^*, pk^*, m_i^*) .
 - The signature σ_i^* generated by $\mathcal{A}_2$ can be verified.

IV. PROPOSED ARCHITECTURE

To better meet the specific requirements of RTS, we present the complete construction of the PFPDS in this section, which includes the following steps:

Setup: As shown in Fig. 3, with input system security parameter $\lambda \in \mathbb{Z}_p^*$, the Authority and the KGC sets the system in the following manner.

- 1) Authority sets the *rules* and outputs a set of policy attributes $(pol_1, pol_2, \dots, pol_n) \in \{0, 1\}^*$, and upload the relevant information of different users' policy sets Pol_i to the blockchain, where $Pol_i = pol_j || \dots || pol_k$ for $pol_j, \dots, pol_k \in \{0, 1\}^*$, $||$ denotes a bitwise concatenation.
- 2) KGC chooses an elliptic curves group $\mathbb{G}$ and a base point G , then it selects $sk_{KGC} \in \mathbb{Z}_p^*$ and calculates $pk_{KGC} = sk_{KGC} \cdot G$ as its public key.
- 3) The Authority picks a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$.
- 4) Finally, KGC maintains sk_{KGC} in secret and discloses the system parameters, $params = \{\mathbb{G}, G, pk_{KGC}, H\}$.

SetSecretValue: To generate X_i , the User chooses a number $x_i \in_R \mathbb{Z}_p^*$ as its secret value and performs the scalar multiplication $X_i = x_i \cdot G$.

SetPartialPrivKey: As shown in Fig. 3, when the User forwards its policy attributes, Pol_i , to the KGC, the KGC follows the ensuing steps to produce the User's partial private key.

- 1) When provided with system parameters, $params$, KGC's master private key, sk_{KGC} , and the User's policy attributes, Pol_i , KGC selects $a_i \in \mathbb{Z}_p^*$ at random and calculates $A_i = a_i \cdot G$.
- 2) KGC computes $h_{1,i} = H(Pol_i || B_i || counter)$, where $B_i = X_i + A_i$, $s_i = a_i + h_{1,i} \cdot sk_{KGC}$ as its partial private key.
- 3) Upon receiving (s_i, A_i) , the User verifies if $s_i \cdot G = A_i + h_{1,i} \cdot pk_{KGC}$ holds true. The partial private key is deemed valid if the equation is satisfied and invalid otherwise.

SetUserKey: To generate their public-private key pair, the User performs the subsequent steps. Specifically, the User sets $sk_i = x_i + s_i$ as its private key and calculates $pk_i = (x_i + s_i) \cdot G$ as public key.

Sign: This stage is shown in Fig. 4, by taking a message $m_i \in \mathbb{Z}_p^*$, the following steps outline the procedure for generating the signature.

- 1) To generate the signature, the User first chooses a number $r_i \in_R \mathbb{Z}_p^*$, and then calculates $R_i = r_i \cdot G$, $h_{2,i} = H(m_i || R_i)$.
- 2) Compute $S_i = r_i + h_{2,i} \cdot sk_i$, and set $\sigma_i' = (R_i, S_i)$.

3) Output the signature $\sigma_i = (\sigma'_i, B_i)$.

Subsequently, the User sends the signature σ_i to a Sensor.

Verify: The verification process is shown in Fig. 4. After receiving the message $\{params, Pol_i, pk_i, m_i, \sigma_i\}$, the Sensor first verifies the User's policy permissions. Specifically, the Sensor queries the policy set information Pol'_i from blockchain. The signatures can only be verified after the User's policy permissions are verified where $Pol'_i = Pol_i$. Then, the Sensor verifies the signature off-chain after receiving the signature σ_i . If the equation $S_i \cdot G = R_i + h_{2,i} \cdot pk_i$ is established, the verification result will be sent to the blockchain.

BatchVerify: Upon receipt of numerous sets of messages $\{params, Pol_i, pk_i, m_i, \sigma_i\}$ from various $User_i$ ($i = 1, 2, \dots, n$), the Sensor proceeds to compute $R = \sum_{i=1}^n R_i$, $S = \sum_{i=1}^n S_i$, $\sigma' = (R, S)$, and outputs aggregated signature $\sigma = (\sigma', B_i)$. In the meantime, the Sensor sends σ and $\{params, Pol_i, pk_i, m_i\}$ ($i = 1, 2, \dots, n$) to other Sensors, and the other Sensors verify it off-chain. Accept σ if the equation $S \cdot G = R + \sum_{i=1}^n h_{2,i} \cdot pk_i$ holds, then the verification result will be sent to the blockchain; if not, will reject the message.

V. SECURITY ANALYSIS

This section aims to demonstrate the security of the proposed scheme under the ROM by leveraging the ECDLP assumption discussed in Section II and the adversarial model presented in Section III.

A. Formal Security Analysis

Theorem 1: The proposed PFPDS scheme is Existentially Unforgeable under the adaptive Chosen Message Attacks (EUF-CMA) against two types of adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$ defined previously.

To provide proof of security for the proposed scheme, the proof utilizes Lemmas 1 and 2.

Lemma 1: Suppose that there is a Probabilistic Polynomial Time (PPT) adversary $\mathcal{A}_1$ that can break $(t, Q_{psk}, Q_{sk}, Q_{sig}, \varepsilon)$ -security against EUF-CMA with a non-negligible advantage ε , then there exists a PPT challenger $\mathcal{C}_1$ that can solve the ECDLP problem with time t' and non-negligible advantages ε' , where

$$\varepsilon' \geq \left(1 - \frac{Q_{psk} + Q_{sk} + Q_{sig}}{n}\right) \cdot \varepsilon^5,$$

$$t' \geq t + (Q_{psk} + Q_{sk} + Q_{sig}) \cdot T.$$

Proof: Assuming that the adversary $\mathcal{A}_1$ can break the PFPDS scheme with a random ECDLP instance: given G , $Q = s \cdot G$, the challenger $\mathcal{C}_1$'s goal is to obtain a solution s of ECDLP through interactions with $\mathcal{A}_1$. For this purpose, $\mathcal{C}_1$ picks Pol_i^* as the challenge policy.

1) *Initialization:* $\mathcal{C}_1$ executes the *Setup* algorithm to initialize system parameters $params = \{G, G, pk_{KGC}, H\}$, sets $pk_{KGC} = Q = s \cdot G$, where Q is a random ECDLP instance, and then sends $params$ to $\mathcal{A}_1$.

2) *Queries:* $\mathcal{C}_1$ generates an initially empty list $\mathcal{L} = (Pol_i, \beta_i, A_i, x_i, pk_i, sk_i)$ and provides $\mathcal{A}_1$ with query to the following oracles.

a) *H queries:* At the beginning, $\mathcal{C}_1$ maintains two lists, $L_{H1}: (Pol_i, B_i, h_{1,i})$, $L_{H2}: (Pol_i, m_i, R_i, h_{2,i})$, and sets both to be empty.

Upon receiving a query for (Pol_i, B_i) , $\mathcal{C}_1$ checks if it exists in the L_{H1} list. If it does, $\mathcal{C}_1$ returns $h_{1,i}$ to $\mathcal{A}_1$. If it is nonexistent, $\mathcal{C}_1$ produces a random number $h_{1,i} \in \mathbb{Z}_p^*$ and sets $h_{1,i} = H(Pol_i || B_i || counter)$. Subsequently, $\mathcal{C}_1$ returns $h_{1,i}$ to $\mathcal{A}_1$.

When $\mathcal{C}_1$ receives a query for (Pol_i, m_i, R_i) , it checks if it exists in the L_{H2} list. If it does, $\mathcal{C}_1$ returns $h_{2,i}$ to $\mathcal{A}_1$. If it is nonexistent, $\mathcal{C}_1$ generates a random number $h_{2,i} \in \mathbb{Z}_p^*$ and sets $h_{2,i} = H(m_i || R_i)$. $\mathcal{C}_1$ then returns $h_{2,i}$ to $\mathcal{A}_1$.

Finally, $\mathcal{C}_1$ adds the tuple $(Pol_i, B_i, h_{1,i})$ to the list L_{H1} and $(Pol_i, m_i, R_i, h_{2,i})$ to the list L_{H2} .

b) *Partial private key queries:* $\mathcal{A}_1$ makes Q_{psk} partial private key queries to $\mathcal{C}_1$. If $Pol_i = Pol_i^*$, this simulation is aborted, and $\mathcal{C}_1$ returns " $\perp$ ". Else if $Pol_i \neq Pol_i^*$, $\mathcal{C}_1$ checks the list $\mathcal{L}$, if Pol_i exists, returns the tuple (Pol_i, β_i, A_i) to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ picks two numbers $\alpha_i, \beta_i \in_R \mathbb{Z}_p^*$ and sets $s_i = \beta_i$, $H(Pol_i || B_i || counter) = h_{1,i} = \alpha_i$. Then it computes $A_i = \beta_i \cdot G - \alpha_i \cdot pk_{KGC}$, where (β_i, A_i) is the partial private key. $\mathcal{C}_1$ then sends (β_i, A_i) to $\mathcal{A}_1$ and appends the tuple $(Pol_i, \beta_i, A_i, *, *, *)$ to the list $\mathcal{L}$. The correctness of partial private key (β_i, A_i) as

$$\begin{aligned} s_i \cdot G &= \beta_i \cdot G \\ &= A_i + \alpha_i \cdot pk_{KGC} \\ &= A_i + h_{1,i} \cdot pk_{KGC} \end{aligned}$$

c) *Public key queries:* When $\mathcal{A}_1$ requests the public key Q_{pk} for the policy set Pol_i , $\mathcal{C}_1$ checks whether Pol_i exists in the list $\mathcal{L}$. If it does, $\mathcal{C}_1$ sends the corresponding pk_i to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ retrieves the tuple $(Pol_i, B_i, h_{1,i})$ from L_{H1} , then selects a random value $x_i \in_R \mathbb{Z}_p^*$ and computes $X_i = x_i \cdot G$, $B_i = X_i + A_i$. Finally, $\mathcal{C}_1$ sets $pk_i = B_i + h_{1,i} \cdot pk_{KGC}$ as the public key and transmits it to $\mathcal{A}_1$. Additionally, $\mathcal{C}_1$ updates $(Pol_i, \beta_i, A_i, x_i, pk_i, *)$ in the $\mathcal{L}$ list.

d) *Private key queries:* $\mathcal{A}_1$ makes Q_{sk} queries to $\mathcal{C}_1$. If $Pol_i = Pol_i^*$, $\mathcal{C}_1$ terminates the simulation and returns " $\perp$ " to $\mathcal{A}_1$. If $Pol_i \neq Pol_i^*$, $\mathcal{C}_1$ computes $sk_i = x_i + \beta_i$, then updates the list $\mathcal{L} = (Pol_i, \beta_i, A_i, x_i, pk_i, sk_i)$.

e) *Public key replace queries:* $\mathcal{A}_1$ replaces the public key pk_i with pk'_i , and sends pk'_i to $\mathcal{C}_1$.

f) *Signing queries:* If $\mathcal{A}_1$ requests the Q_{sig} signature for (Pol_i, m_i) and $Pol_i \neq Pol_i^*$, running the *Sign* algorithm, $\mathcal{C}_1$ produces the signature σ_i and transmits it to $\mathcal{A}_1$. If $Pol_i = Pol_i^*$, $\mathcal{C}_1$ recovers the list $\mathcal{L} = (Pol_i, \beta_i, A_i, x_i, pk_i, sk_i)$ and computes $B_i = x_i \cdot G + A_i$, subsequently, chooses random numbers $r_i, \theta_i, \varphi_i \in \mathbb{Z}_p^*$. $\mathcal{C}_1$ sets $R_i = r_i \cdot G$,

$h_{2,i} = H(m_i || R_i) = \varphi_i$, and computes $sk_i = x_i + \beta_i$, $S_i = r_i + \varphi_i \cdot sk_i$. Finally, $\mathcal{C}_1$ sets $\sigma'_i = (R_i, S_i)$ and outputs the signature $\sigma_i = (\sigma'_i, B_i)$. The correctness of σ_i as

$$\begin{aligned} S_i \cdot G &= (r_i + \varphi_i \cdot sk_i) \cdot G \\ &= (r_i + \varphi_i \cdot (x_i + \beta_i)) \cdot G \\ &= R_i + \varphi_i \cdot (X_i + A_i + \alpha_i \cdot pk_{KGC}) \\ &= \varphi_i \cdot (B_i + \alpha_i \cdot pk_{KGC}) + R_i \\ &= \varphi_i \cdot pk_i + R_i \end{aligned}$$

- 3) *Forgery*: At the end of the process, $\mathcal{A}_1$ terminates the query and produces a forged signature $\sigma_i^* = (\sigma_i'^*, B_i^*)$ as the output. While aggregating the signatures, it is not mandatory for all the signatures to be forged. However, the forged aggregated signature must include the signature of the target identity Pol_i^* . In case the signature of Pol_i^* is not forged, $\mathcal{A}_1$ will terminate the simulation and report an “ERROR”. Based on the Forking Lemma [39], using the same random tape but distinct hash values $h_{1,i}$, $\mathcal{A}_1$ can create two legitimate signatures σ_i^{*I} and σ_i^{*II} . For the sake of simplification, let us assume that $Pol_i^* = Pol_1^*$. After that we have

$$\sigma^{*I} = \sum_{i=2}^n \sigma_i^{*I} + \sigma_1^{*I}, \quad \sigma^{*II} = \sum_{i=2}^n \sigma_i^{*II} + \sigma_1^{*II}$$

More specifically,

- a) In the execution of $\mathcal{C}_1$, $\mathcal{A}_1$ outputs a valid forgery $\sigma_1^{*I} = ((R_1^{*I}, \widetilde{S_1^{*I}}), B_i^*)$, we have

$$\widetilde{S_1^{*I}} \cdot G = \widetilde{h_{2,i}^I} \cdot pk_i^I + R_1^{*I}. \quad (1)$$

- b) In the execution of $\mathcal{C}_1$, $\mathcal{A}_1$ outputs a valid forgery $\sigma_1^{*I} = ((R_1^{*I}, \widetilde{S_1^{*I}}), B_i^*)$, we have

$$\widetilde{S_1^{*I}} \cdot G = \widetilde{h_{2,i}^I} \cdot pk_i^I + R_1^{*I}. \quad (2)$$

- c) In the execution of $\mathcal{C}_1$, $\mathcal{A}_1$ outputs a valid forgery $\sigma_1^{*II} = ((R_1^{*II}, \widetilde{S_1^{*II}}), B_i^*)$, we have

$$\widetilde{S_1^{*II}} \cdot G = \widetilde{h_{2,i}^{II}} \cdot pk_i^{II} + R_1^{*II}. \quad (3)$$

- d) In the execution of $\mathcal{C}_1$, $\mathcal{A}_1$ outputs a valid forgery $\sigma_1^{*II} = ((R_1^{*II}, \widetilde{S_1^{*II}}), B_i^*)$, we have

$$\widetilde{S_1^{*II}} \cdot G = \widetilde{h_{2,i}^{II}} \cdot pk_i^{II} + R_1^{*II}. \quad (4)$$

From (1) and (2), we have $pk_i^I = B_i^* + h_{1,i}^I \cdot pk_{KGC}$, then we can calculate

$$\begin{aligned} (\widetilde{S_1^{*I}} - \widetilde{S_1^{*II}}) \cdot G &\cdot (\widetilde{h_{2,i}^I} - \widetilde{h_{2,i}^{II}})^{-1} \\ &= B_i^* + h_{1,i}^I \cdot pk_{KGC}. \end{aligned} \quad (5)$$

Similarly, from (3) and (4), we have

$$(\widetilde{S_1^{*II}} - \widetilde{S_1^{*I}}) \cdot G \cdot (\widetilde{h_{2,i}^{II}} - \widetilde{h_{2,i}^I})^{-1}$$

$$= B_i^* + h_{1,i}^{II} \cdot pk_{KGC}. \quad (6)$$

Solving for pk_{KGC} in (5) and (6), we eventually have

$$\begin{aligned} &\left\{ (\widetilde{S_1^{*I}} - \widetilde{S_1^{*II}}) \cdot (\widetilde{h_{2,i}^I} - \widetilde{h_{2,i}^{II}})^{-1} - (\widetilde{S_1^{*II}} - \widetilde{S_1^{*I}}) \right. \\ &\quad \left. \cdot (\widetilde{h_{2,i}^{II}} - \widetilde{h_{2,i}^I})^{-1} \right\} \cdot (h_{1,i}^I - h_{1,i}^{II})^{-1} \cdot G = pk_{KGC}. \end{aligned} \quad (7)$$

Because $pk_{KGC} = s \cdot G$, we finally get the solution of the ECDLP problem as s . $\square$

After that, we examine the three events required for the success of $\mathcal{C}_1$:

E_1 : $\mathcal{C}_1$ does not terminate during $\mathcal{A}_1$'s inquiries.

E_2 : $\mathcal{A}_1$ has the ability to generate a valid forged signature.

E_3 : The forged signature of $\mathcal{A}_1$ is valid.

$\mathcal{C}_1$ successfully breaks the ECDLP problem if all of these events occur. The probability $\Pr[E_1 \wedge E_2 \wedge E_3]$ is

$$\Pr[E_1 \wedge E_2 \wedge E_3] = \Pr[E_1] \cdot \Pr[E_2] \cdot \Pr[E_3].$$

Regarding E_1 , we have divided it into three parts, where $\frac{1}{n}$ represents the probability that $\mathcal{A}_1$'s query is the target user.

For the *SetPartialPrivKey* algorithm, the probability that $\mathcal{C}_1$ does not terminate is

$$(1 - \frac{1}{n})^{Q_{psk}} \geq (1 - \frac{Q_{psk}}{n}).$$

Similarly, as to *SetUserKey* algorithm, we have

$$(1 - \frac{1}{n})^{Q_{sk}} \geq (1 - \frac{Q_{sk}}{n}).$$

With regard to *Sign* algorithm, we can get

$$(1 - \frac{1}{n})^{Q_{sig}} \geq (1 - \frac{Q_{sig}}{n}).$$

To sum up, the total probability of E_1 is

$$\Pr[E_1] \geq \left(1 - \frac{Q_{psk} + Q_{sk} + Q_{sig}}{n}\right).$$

In the matter of E_2 , assuming that $\mathcal{A}_1$'s advantage in successfully forging a signature is ε , then $\Pr[E_2] \geq \varepsilon$; for E_3 , $\mathcal{A}_1$ returns a valid signature. If it is possible to authenticate the signature $\mathcal{C}_1$ can use it to solve the ECDLP problem, the probability is $\Pr[E_3] \geq \varepsilon^4$.

$\mathcal{C}_1$ can break the ECDLP problem with advantages of ε' is

$$\varepsilon' \geq \left(1 - \frac{Q_{psk} + Q_{sk} + Q_{sig}}{n}\right) \cdot \varepsilon^5.$$

The time complexity of $\mathcal{C}_1$ is equivalent to that of $\mathcal{A}_1$ plus the duration needed to answer $(Q_{psk} + Q_{sk})$ public/private key queries along with Q_{sig} signature queries. Set T as the time cost for calculating the indexation. Consequently, the overall time required by $\mathcal{C}_1$ to solve the ECDLP problem is

$$t' \geq t + (Q_{psk} + Q_{sk} + Q_{sig}) \cdot T.$$

Lemma 2: Suppose that there is a PPT adversary $\mathcal{A}_2$ that can break $(t, \mathcal{Q}_{psk}, \mathcal{Q}_{sk}, \mathcal{Q}_{sig}, \varepsilon)$ -security against EUF-CMA with a non-negligible advantage ε , then there exists a PPT challenger $\mathcal{C}_2$ that can solve the ECDLP problem with time t' and non-negligible advantages ε' , where

$$\varepsilon' \geq \left(1 - \frac{\mathcal{Q}_{psk} + \mathcal{Q}_{sk} + \mathcal{Q}_{sig}}{n}\right) \cdot \varepsilon^5,$$

$$t' \geq t + (\mathcal{Q}_{psk} + \mathcal{Q}_{sk} + \mathcal{Q}_{sig}) \cdot T.$$

Proof: Assuming that the adversary $\mathcal{A}_2$ can break the PFPDS scheme with a random ECDLP instance: given G , $X_i = x_i \cdot G$, the challenger $\mathcal{C}_2$'s goal is to obtain a solution x_i of ECDLP through interactions with $\mathcal{A}_2$. For this purpose, $\mathcal{C}_2$ picks Pol_i^* as the challenge policy.

- 1) *Initialization:* $\mathcal{C}_2$ executes the *Setup* algorithm to initialize system parameters $params = \{\mathbb{G}, G, pk_{KGC}, H\}$, sets $pk_{KGC} = Q = s \cdot G$, wherein, $s \in \mathbb{Z}_p^*$ is a random number, and then transmits both $params$ and s to $\mathcal{A}_2$.
- 2) *Queries:* $\mathcal{C}_2$ generates an initially empty list $\mathcal{L} = (Pol_i, s_i, A_i, x_i, pk_i, sk_i)$ and provides $\mathcal{A}_2$ with query to the following oracles.

- a) *H queries:* At the beginning, $\mathcal{C}_2$ maintains two lists, $L_{H_1}: (Pol_i, B_i, h_{1,i})$, $L_{H_2}: (Pol_i, m_i, R_i, h_{2,i})$, and sets both to be empty.

When $\mathcal{C}_2$ receives a query for (Pol_i, B_i) , it verifies if it exists in the list L_{H_1} . If it is present, $\mathcal{C}_2$ returns $h_{1,i}$ to $\mathcal{A}_2$. If it is not present, $\mathcal{C}_2$ generates a random number $h_{1,i} \in \mathbb{Z}_p^*$ and sets $h_{1,i} = H(Pol_i || B_i || counter)$. Subsequently, $\mathcal{C}_2$ returns $h_{1,i}$ to $\mathcal{A}_2$.

When $\mathcal{C}_2$ receives the query for (Pol_i, m_i, R_i) , it checks if it exist in the L_{H_2} list. If it does, $\mathcal{C}_2$ returns $h_{2,i}$ to $\mathcal{A}_2$. If it is nonexistent, $\mathcal{C}_2$ generates a random number $h_{2,i} \in \mathbb{Z}_p^*$ and sets $h_{2,i} = H(m_i || R_i)$. $\mathcal{C}_2$ then returns $h_{2,i}$ to $\mathcal{A}_2$.

Finally, $\mathcal{C}_2$ adds the tuple $(Pol_i, B_i, h_{1,i})$ to the list L_{H_1} and $(Pol_i, m_i, R_i, h_{2,i})$ to the list L_{H_2} .

- b) *Partial private key queries:* $\mathcal{A}_2$ makes $\mathcal{Q}_{psk}$ partial private key queries to $\mathcal{C}_2$. If $Pol_i = Pol_i^*$, this simulation is aborted, and $\mathcal{C}_2$ returns “ $\perp$ ”. Else if $Pol_i \neq Pol_i^*$, $\mathcal{C}_2$ checks the list $\mathcal{L}$, if Pol_i exists, returns the tuple (Pol_i, β_i, A_i) to $\mathcal{A}_2$. Otherwise, $\mathcal{C}_2$ picks $a_i, \alpha_i \in_R \mathbb{Z}_p^*$, sets $A_i = a_i \cdot G$, $H(Pol_i || B_i || counter) = h_{1,i} = \alpha_i$, and then computes $s_i = a_i + \alpha_i \cdot s$, where (s_i, A_i) is the partial private key. At the end, $\mathcal{C}_2$ sends (s_i, A_i) to $\mathcal{A}_2$ and appends the tuple $(Pol_i, s_i, A_i, *, *, *)$ to $\mathcal{L}$.

The correctness of partial private key (s_i, A_i) as

$$\begin{aligned} s_i \cdot G &= (a_i + \alpha_i \cdot s) \cdot G \\ &= A_i + \alpha_i \cdot s \cdot G \\ &= A_i + h_{1,i} \cdot pk_{KGC} \end{aligned}$$

- c) *Public key queries:* When $\mathcal{A}_2$ requests the public key $\mathcal{Q}_{pk}$ for the policy set Pol_i , $\mathcal{C}_2$ checks whether Pol_i exists in the list $\mathcal{L}$. If it does, $\mathcal{C}_2$ returns the

corresponding pk_i to $\mathcal{A}_2$. Otherwise, if $Pol_i = Pol_i^*$, $\mathcal{C}_2$ recovers the tuple $(Pol_i, B_i, h_{1,i})$ from L_{H_1} then calculates $B_i = X_i + A_i$, $x_i = \perp$, sets the public key $pk_i = B_i + h_{1,i} \cdot pk_{KGC}$, sends pk_i to $\mathcal{A}_2$ and then updates $(Pol_i, s_i, A_i, \perp, pk_i, *)$ to $\mathcal{L}$. If $Pol_i \neq Pol_i^*$, $\mathcal{C}_2$ randomly selects a number $x_i \in \mathbb{Z}_p^*$, sets $pk_i = (x_i + s_i) \cdot G$, and inserts into the list $\mathcal{L} = (Pol_i, s_i, A_i, x_i, pk_i, *)$.

- d) *Private key queries:* $\mathcal{A}_2$ makes $\mathcal{Q}_{sk}$ queries to $\mathcal{C}_2$. $\mathcal{C}_2$ aborts this simulation and returns “ $\perp$ ” to $\mathcal{A}_2$ as if $Pol_i = Pol_i^*$. If $Pol_i \neq Pol_i^*$, $\mathcal{C}_2$ computes $sk_i = x_i + s_i$, then updates the list $\mathcal{L} = (Pol_i, s_i, A_i, x_i, pk_i, sk_i)$.
- e) *Signing queries:* If $\mathcal{A}_2$ requests the $\mathcal{Q}_{sig}$ signature for (Pol_i, m_i) and $Pol_i \neq Pol_i^*$, $\mathcal{C}_2$ executes the *Sign* algorithm to produce the signature σ_i and sends it to $\mathcal{A}_2$. If $Pol_i = Pol_i^*$, $\mathcal{C}_2$ recovers the list $\mathcal{L} = (Pol_i, s_i, A_i, x_i, pk_i, sk_i)$ and computes $B_i = X_i + A_i$, subsequently, chooses random numbers $r_i, \theta_i, \varphi_i \in \mathbb{Z}_p^*$. $\mathcal{C}_2$ sets $R_i = r_i \cdot G$, $h_{2,i} = H(m_i || R_i) = \varphi_i$, and computes $S_i = r_i + \varphi_i \cdot sk_i$. Finally, $\mathcal{C}_2$ sets $\sigma'_i = (R_i, S_i)$ and outputs the signature $\sigma_i = (\sigma'_i, B_i)$.

The correctness of σ_i as

$$\begin{aligned} S_i \cdot G &= (r_i + \varphi_i \cdot sk_i) \cdot G \\ &= (r_i + \varphi_i \cdot (x_i + s_i)) \cdot G \\ &= R_i + \varphi_i \cdot (X_i + A_i + \alpha_i \cdot pk_{KGC}) \\ &= \varphi_i \cdot (B_i + \alpha_i \cdot pk_{KGC}) + R_i \\ &= \varphi_i \cdot pk_i + R_i \end{aligned}$$

- 3) *Forgery:* At the end of the process, $\mathcal{A}_2$ terminates the query and produces a forged signature $\sigma_i^* = (\sigma'_i, B_i^*)$ as the output. While aggregating the signatures, it is not mandatory for all the signatures to be forged. However, the forged aggregated signature must include the signature of the target identity Pol_i^* . In the case the signature of Pol_i^* is not forged, $\mathcal{A}_2$ will terminate the simulation and outputs an “ERROR”. Based on the Forking Lemma [39], using the same random tape but distinct hash values $h_{1,i}$, $\mathcal{A}_2$ can create two legitimate signatures σ_i^{*I} and σ_i^{*II} . For the sake of simplification, we assume that $Pol_i^* = Pol_1^*$. After that we have

$$\sigma^{*I} = \sum_{i=2}^n \sigma_i^{*I} + \sigma_1^{*I}, \quad \sigma^{*II} = \sum_{i=2}^n \sigma_i^{*II} + \sigma_1^{*II}$$

More specifically, we have

$$S_1^{*I} \cdot G = h_{2,i}^I \cdot pk_i^* + R_1^*. \quad (8)$$

$$S_1^{*II} \cdot G = h_{2,i}^{II} \cdot pk_i^* + R_1^*. \quad (9)$$

From (8) and (9), we have $pk_i^* = B_i^* + h_{1,i} \cdot pk_{KGC}$, $B_i = X_i + A_i$, then we can calculate

$$\begin{aligned} X_i &= (h_{2,i}^{II} - h_{2,i}^I)^{-1} \cdot (S_1^{*II} - S_1^{*I}) \\ &\quad \cdot G - A_i - h_{1,i} \cdot s \cdot G \end{aligned} \quad (10)$$

Because $X_i = x_i \cdot G$, we finally get the solution of the ECDLP problem as x_i . $\square$

The probability that $\mathcal{A}_2$ can successfully solve the ECDLP problem and the total time cost are the same as in the proof of Lemma 1.

B. Other Security Requirements

The proposed PFPDS scheme also implements other security requirements in the communication process.

- 1) *Unforgeability*: Based on Lemmas 1 and 2 in the security proof mentioned above, we have proven the existential unforgeability of our PFPDS scheme under chosen message attacks. These adversaries are unable to forge valid signatures that satisfy $S_i \cdot G = R_i + h_{2,i} \cdot pk_i$.
- 2) *Message Authentication and Integrity*: In our scheme, every message generated by the train needs to be signed before being sent to nearby sensors. Sensors can verify the integrity and validity of the message by checking if the condition $S_i \cdot G = R_i + h_{2,i} \cdot pk_i$ holds. This ensures that the message has not been tampered with or forged by attackers or unauthorized trains. Therefore, the proposed scheme meets the security requirements for message authentication and integrity.
- 3) *Anonymity*: According to our proposal, the true identity of the train is completely anonymous. Except for the SA, other trains and sensors have no knowledge of the train's real identity. During the communication process, the true identity of BCUser is concealed within the policy and BCUsers communicate with each other using the policies generated by the SA. Except for the SA, no adversary can obtain the real identity of BCUser from the policy set.
- 4) *Unlinkability*: As per our scheme, the train signs and broadcasts messages. Due to the random nature of the r_i value in the signature, adversaries cannot link two messages to the same user. Therefore, the proposed PFPDS scheme is unlinkable.
- 5) *Attack Resistance*: This scheme also meets some other anti attack requirements.

Man-in-the-Middle Attacks: In our proposed scheme, if a malicious user intercepts a message $\{params, Pol_i, pk_i, m_i, \sigma_i\}$, it is not possible to modify it, because through the check condition $S_i \cdot G = R_i + h_{2,i} \cdot pk_i$, which can be easily detected by sensors.

Impersonation Attacks: Within our framework, upload policy set information to blockchain. Before communication, users query the blockchain to verify the legitimacy of the policy set information. If a malicious user tries to impersonate a legitimate BCUser using policy information, it is infeasible on the blockchain.

VI. APPLICATION FOR RAIL TRANSIT SYSTEMS

We have defined our system structure in Section III, and in this Section, we will delve deeper into the system architecture and present our suggested security protocol, along with its practical implementation in RTS scenarios.

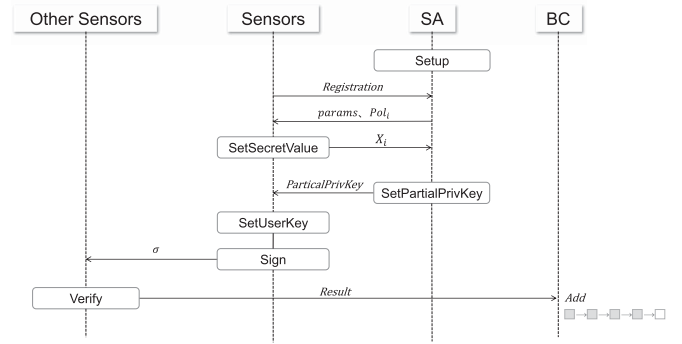


Fig. 5. Security protocol for Rail Transit Systems.

A. Protocol

This section will cover the introduction of a security protocol that showcases the functioning of our system. To simplify the representation, we divide the system into four entities, namely SA, BC, BCUser, and CS. Each entity consists of a set of components with specific functions. In our proposed PFPDS system, there are the following entities: NC, Authority, and KGC, to simplify matters, we shall henceforth refer to these entities as SA. The NC is responsible for defining rules, while the Authority generates a set of policies based on these rules. Simultaneously, the SA initializes the system, generates the master key, and defines the public parameters of the system. In addition, the BCUser comprises onboard units, sensors on both sides of the track, track workers, and train drivers who received public-private pair keys from the KGC. All communication requirements of our protocol are completed by CS.

The steps involved in the protocol are depicted in Fig. 5 and will be expounded upon in the following sections.

- Initially, the SA is required to establish the system by producing the system parameters, denoted as $params$.
- Users are required to register with the Authority, which then assigns distinct policies according to their respective permissions and archives them in their databases.
- Partial private key $ParticalPrivKey$ is generated by KGC with corresponding policies and unique counters. Then, KGC sends $ParticalPrivKey$ and $params$ to the User. The User chooses a confidential value and merges it with $ParticalPrivKey$ to generate the User's private key, then calculates its public key using the private key and public parameters.
- During the communication phase, the user generates PFPDS and sends it to other Sensors for verification.
- Only after the signature is verified will the Sensor transmit the verification results to the blockchain and store them.
- The system will coordinate PFPDS.

B. Application

Traditional RTS, completely dependent on AMS, other users cannot verify the signature data, so there is only one trust point, which means that the only data verification is done by NC. But in our system, any other Sensors can verify the signature. The

following describes the practical application based on the secure communication protocol.

Our scheme differs from traditional blockchain applications by providing policy-based regulation of users, with each user possessing permissions allocated by an Authority. This feature is useful in RTS, where tasks can be safely performed by authorized users, and in fact, is useful in many possible scenarios, two scenarios of which we will consider in this article. The first scenario is the authentication of the driver before operating the train, and the second scenario is the batch verification of the large amount of traffic information received by the sensors.

For example, train abc is in charge of a group of drivers for which the authority assigns the policy set Pol_{abc} , which means that both train abc and drivers $(d_1, d_2, \dots, d_n)$ belong to the policy set Pol_{abc} . In our scheme, the KGC is only required to produce the partial private key once for the policy set Pol_{abc} . It is worth noting that in the policy set Pol_{abc} , each driver selects a different secret value and counter, so a different driver based on the partial private key generates different public-private key pairs. In specific terms, when the driver d_2 starts duty, the verification results of σ_{d_1} should already exist in the blockchain. d_2 first queries the verification result of σ_{d_1} in the blockchain, if it exists, d_2 sends the signature of the on-duty request message, $\sigma_{d_2} = ((R_{d_2}, S_{d_2}), B_i)$, and then the Sensor verifies σ_{d_2} . If the verification is successful, driver d_2 allows the train to be driven while the verification results are sent to the blockchain. Similarly, when d_3 is on duty, the blockchain needs to have the verification result of σ_{d_2} , and σ_{d_1} should also be reconciled by the miner. Above all, the LPDB can be coordinated for all users.

Correspondingly, the process of the TS sending messages and verifying the messages is similar to that of a driver driving a train. Let us take a brief look at the application of the TS. Each sensor in the TS is placed in sequence, therefore PFPDS signature is to comply with the provisions of the given sequence.

During the blockchain coordination process, miners are responsible for verifying whether the driver has the necessary authorization to operate the train by examining whether the PFPDS signature execution process falls under the policy set Pol_{abc} . Thus, our system guarantees that only authorized drivers are permitted to operate trains.

Similarly, let us now take a look at the application of the proposed scheme in the second scenario. In the RTS, there is an increasing number of Sensor devices generating and sharing a large amount of data. This results in a significant volume of traffic information that needs to be transmitted. If each Sensor individually verifies every piece of information, it would incur an overwhelming computational cost and fail to meet the low-latency requirements of real-time systems. Therefore, the batch verification algorithm proposed in this article can be effectively applied to RTS.

VII. PERFORMANCE EVALUATION

We have implemented our approach using C language and MIRACL libraries on a personal computer with the following specifications: Lenovo Intel (R) Core (TM) i7-10700 CPU @ 2.90 GHz 2.90 GHz, 16.0 GB RAM, Windows 10 21H2.

TABLE II
NOTATION AND UPTIME

Notation	Operation	Uptime (ms)
T_a	Adding Points on Elliptic Curves	0.018
T_m	Point multiplication operation on Elliptic Curves	6.603
T_e	Modular exponentiation on Elliptic Curves	12.529
T_H	Hash function	0.02
T_{HTP}	Hash mapping to points	0.603
T_P	Bilinear pairing operation	21.892

TABLE III
COMPARISONS OF SIGN-LENGTH AND ASSUMPTION

Scheme	Length of Signature	Assumption
Wang et al. [18]	$ \mathbb{G} + \mathbb{Z}_p $	$\times$
Sun et al. [17]	$3 \mathbb{G} $	$\times$
Lin et al. [27]	$n \mathbb{G} + \mathbb{Z}_p $	CDHP
Mu et al. [9]	$ \mathbb{G} + \mathbb{Z}_p $	q-BSDH
Our PFPDS	$2 \mathbb{Z}_p $	ECDLP

This implementation serves as a testament to the practicality and efficacy of our scheme. For operations based on bilinear pairings and elliptic curves, we set the security strength to 128 bits, and choose Type-I pairings for bilinear pairings $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$. Group $\mathbb{G}_1$ is an additive cyclic group, while $\mathbb{G}_T$ is a multiplicative cyclic group. As shown in Table II, we tested 1000 rounds of different operations in the MIRACL library and used their average as the final result. We conducted a comparative analysis between our PFPDS scheme, the ABS schemes proposed by Wang et al. [18], Sun et al. [17], and Lin et al. [27], as well as the PDS scheme introduced by Mu et al. [9]. Our comparison focused on various aspects such as signature length, signature generation cost, verification cost, and complexity assumptions for each scheme.

A. Sign-Length and Assumption Analysis

As depicted in Table III, Type-I pairing is formed on a supersingular elliptic curve on F_p , with further elaboration provided therein. Wang et al. [18] scheme's signature length is $|\mathbb{G}| + |\mathbb{Z}_p|$. The signature length of Sun et al. [17] is $3|\mathbb{G}|$. In addition, Lin et al. [27] demonstrated their scheme based on CDHP, with a signature length of $n|\mathbb{G}| + |\mathbb{Z}_p|$. Mu et al. [9] proposed a scheme wherein the signature length is $|\mathbb{G}| + |\mathbb{Z}_p|$, and the security assumption is q-BSDH. Our proposed PFPDS scheme is established on the ECDLP assumption and is accompanied by a security proof, with the signature length of $2|\mathbb{Z}_p|$.

It is evident that our scheme has shorter signature lengths and incurs lower communication costs when compared to the alternative.

B. Computation Cost Analysis

Wang et al. [18]'s scheme requires 2 hash functions, 6 modular exponentiation operations, 2 point multiplication operations on elliptic curves, and one point addition for signature generation, with a total cost of approximately $2T_H + 6T_e + 2T_m + T_a \approx$

TABLE IV
COMPARISONS OF SIGNATURE AND VERIFICATION

Scheme	Cost of Signature	Cost of Verification
Wang et al. [18]	$2T_H + 6T_e + 2T_m + T_a$	$5T_p + 3T_e + 2T_i + 2T_H + 2T_m$
Sun et al. [17]	$5T_p + 4T_H + 7T_e + T_i$	$2T_p + T_i + 3T_e + T_H + T_m$
Lin et al. [27]	$T_H + lT_e + lT_m$	$T_H + lT_e + 3T_p + lT_m$
Mu et al. [9]	T_e	$2T_p$
Our PFPDS	$T_m + T_a$	$2T_m + T_a$

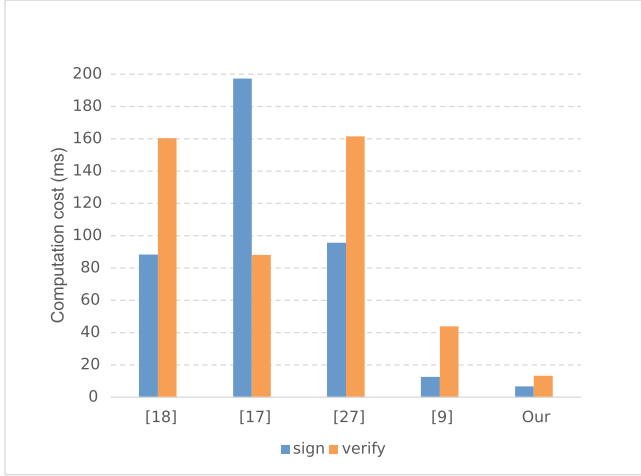


Fig. 6. Comparison of computational costs for signature and verification in a single operation.

88.402 ms, and for the convenience of unified calculation, we take the value of l in the linear relationship as 5. The signature verification requires 5 bilinear pairing operations, 3 modular exponentiation operations, 2 modular inverse operations, 2 hash functions, and 2 point multiplication operations on elliptic curves, with a total cost of $5T_p + 3T_e + 2T_i + 2T_H + 2T_m \approx 160.293$ ms. Similarly, as shown in Table IV, we computed the computational expenses of the signature generation and verification process for the scheme proposed in [9], [17], [27]. As shown in Fig. 6, it is apparent that our scheme incurs less computational overhead in comparison to other schemes. Compared with Wang et al. [18], Sun et al. [17], Lin et al. [27], and Mu et al. [9], our scheme reduces signature generation costs by 92.51%, 96.64%, 93.08%, and 47.15%, respectively. In the same way, our scheme also demonstrates significant advantages in terms of verification costs compared to Wang et al. [18], Sun et al. [17], Lin et al. [27], and Mu et al. [9]. Specifically, our scheme has reduced verification costs by 91.75%, 84.97%, 91.80%, and 69.80% compared to theirs. According to the above analysis, our PFPDS scheme presents notable benefits in terms of both signature generation and verification.

In addition, our scheme is based on Schnorr signatures and can be verified in batches. As shown in Fig. 7, We examine the scenario wherein the Sensor receives a considerable volume of messages (with $n = 100, 200, \dots, 500$), it is clear that the computational overhead of batch verification is approximately 49% higher than that of single verification. Furthermore, it is worth mentioning that the verification duration is directly correlated

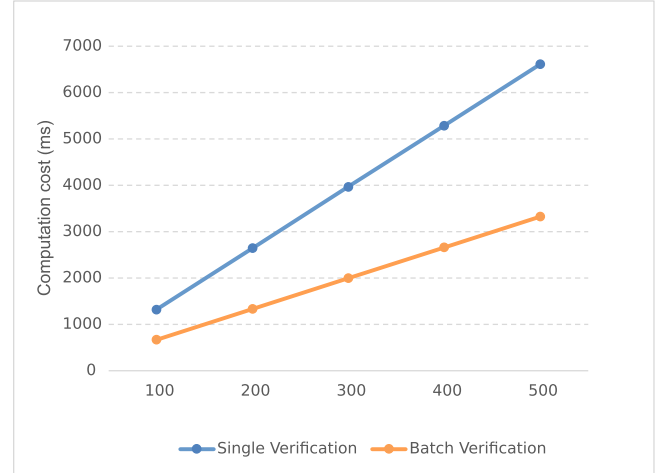


Fig. 7. Comparative analysis of computational expenses for single signature verification and batch verification.

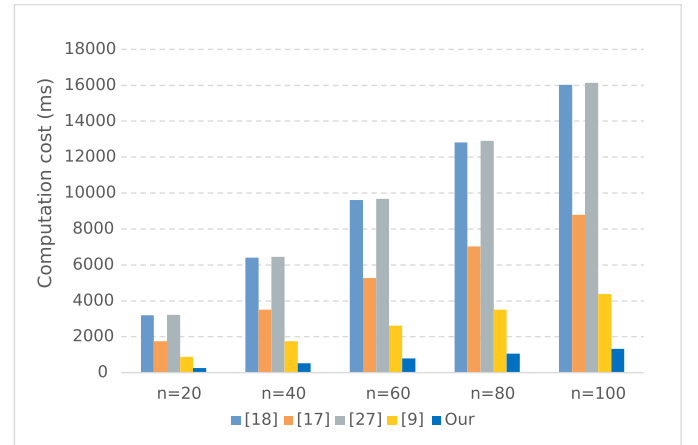


Fig. 8. Comparison of computational costs for batch verification.

with the number of received messages. Therefore, in real-time RTS scenarios, the advantages of our scheme become more evident as the number of communication messages exchanged increases.

On the other hand, in order to better demonstrate the advantages of our scheme, we assume that when $n = 100$, indicating that the sensors receive 100 traffic messages simultaneously. We proceed to calculate the computational overhead required for verifying the aforementioned comparative schemes, and subsequently compare it with our proposed approach. As evidenced by Fig. 8, our scheme provides the capability of batch verification, resulting in significantly reduced computational costs.

VIII. CONCLUSION

The Rail Transit Systems (RTS) is already closely related to our daily lives and work and is one of our most popular transportation systems. As the RTS becomes more popular, there will be higher requirements for its system in terms of high mobility, low latency, data exchange, and data storage.

At the same time, its data security and privacy also face significant challenges. To achieve our vision, we have considered the new Lightweight Policy-Driven Blockchain (LPDB) based on traditional blockchain that can solve most of the problems faced by RTS. What sets this apart from existing schemes is that we provide the Pairing-free Policy-Driven Signature (PFPDS) scheme, which is based on an elliptic curve addition group, avoiding the high computational cost of bilinear pairing, and implementing off-chain signature verification, which can reduce the on-chain overhead of the blockchain. Furthermore, we also take into account the situation where the real-time interactive data volume of the RTS is very large. To address this, we have analyzed the batch verification of the signatures when a Sensor receives a large number of messages. Batch verification can meet the efficient transaction verification needs in blockchain scenarios. In conclusion, we have demonstrated the validity and security of our signature scheme under the ROM and presented experimental findings to corroborate its feasibility.

REFERENCES

- [1] F. Li, K.-Y. Lam, H.-H. Chen, and N. Zhao, "Spectral efficiency enhancement in satellite mobile communications: A game-theoretical approach," *IEEE Wireless Commun.*, vol. 27, no. 1, pp. 200–205, Feb. 2020.
- [2] F. Li, K.-Y. Lam, X. Liu, J. Wang, K. Zhao, and L. Wang, "Joint pricing and power allocation for multibeam satellite systems with dynamic game model," *IEEE Trans. Veh. Technol.*, vol. 67, no. 3, pp. 2398–2408, Mar. 2018.
- [3] R. Hussain and S. Zeadally, "Autonomous cars: Research results, issues, and future challenges," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1275–1313, Secondquarter 2019.
- [4] S. M. Ahsan Kazmi et al., "Infotainment enabled smart cars: A joint communication, caching, and computation approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 9, pp. 8408–8420, Sep. 2019.
- [5] X. Cui and J. Li, "Tools and practices," in *Secure and Trustworthy Transportation Cyber-Physical Systems*. Sun Y. and Song H., Eds. Springer, Singapore: Springer Briefs Comput. Sci., 2017, doi: [10.1007/978-981-10-3892-1_7](https://doi.org/10.1007/978-981-10-3892-1_7).
- [6] J. Kolodziej, C. Hopmann, G. Coppa, D. Grzonka, and A. Widlak, "Intelligent transportation systems - Models, challenges, security aspects," in *Cybersecurity of Digital Service Chains - Challenges, Methodologies, and Tools*. Berlin, Germany: Springer, 2022, pp. 56–82.
- [7] M. B. Mollah et al., "Blockchain for the Internet of Vehicles towards intelligent transportation systems: A survey," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4157–4185, Mar. 2021.
- [8] D. Pulido, G. Darido, R. Munozraskin, and J. Moody, *The Urban Rail Development Handbook*. Washington, DC, USA: World Bank Pub., 2018.
- [9] Y. Mu, F. Rezaeibagha, and K. Huang, "Policy-driven blockchain and its applications for transport systems," *IEEE Trans. Serv. Comput.*, vol. 13, no. 2, pp. 230–240, Mar./Apr. 2020.
- [10] J.-P. Chang et al., "Assessing spatial synergy between integrated urban rail transit system and urban form: A buli-based MCLSGA model with the wisdom of crowds," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 2, pp. 434–448, Feb. 2023.
- [11] G. Costantino, M. De Vincenzi, F. Martinelli, and I. Matteucci, "A privacy-preserving solution for intelligent transportation systems: Private driver DNA," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 258–273, Jan. 2023.
- [12] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," Aug. 21, 2008, doi: [10.2139/ssrn.3440802](https://doi.org/10.2139/ssrn.3440802).
- [13] H. Maji, M. Prabhakaran, and M. Rosulek, "Attribute-based signatures: Achieving attribute-privacy and collusion-resistance," *IACR Cryptol. ePrint Arch.*, p. 328, 2008. [Online]. Available: <https://ia.cr/2008/328>
- [14] S. F. Shahandashti and R. Safavi-Naini, "Threshold attribute-based signatures and their application to anonymous credential systems," in *Proc. 2nd Int. Conf. Cryptol. Afr.*, 2009, pp. 198–216.
- [15] D. Huang and M. Verma, "ASPE: Attribute-based secure policy enforcement in vehicular ad hoc networks," *Ad Hoc Netw.*, vol. 7, no. 8, pp. 1526–1535, 2009.
- [16] R. Guo, H. Shi, Q. Zhao, and D. Zheng, "Secure attribute-based signature scheme with multiple authorities for blockchain in electronic health records systems," *IEEE Access*, vol. 6, pp. 11676–11686, 2018.
- [17] Y. Sun, R. Zhang, X. Wang, K. Gao, and L. Liu, "A decentralizing attribute-based signature for healthcare blockchain," in *Proc. IEEE 27th Int. Conf. Comput. Commun. Netw.*, 2018, pp. 1–9.
- [18] H. Wang and Y. Song, "Secure cloud-based EHR system using attribute-based cryptosystem and blockchain," *J. Med. Syst.*, vol. 42, no. 8, pp. 152:1–152:9, 2018.
- [19] J. Sun, Y. Su, J. Qin, J. Hu, and J. Ma, "Outsourced decentralized multi-authority attribute based signature and its application in IoT," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 1195–1209, Jul.–Sep. 2021.
- [20] Y. Wang, C. Chen, Z. Chen, and J. He, "Attribute-based user revocable data integrity audit for Internet-of-Things devices in cloud storage," *Secur. Commun. Netw.*, vol. 2020, pp. 8837456:1–8837456:10, 2020.
- [21] C. Huang et al., "Toward security as a service: A trusted cloud service architecture with policy customization," *J. Parallel Distrib. Comput.*, vol. 149, pp. 76–88, 2021.
- [22] F. Luo, S. Al-Kuwari, W. Susilo, and D. H. Duong, "Attribute-based proxy re-signature from standard lattices and its applications," *Comput. Stand. Interfaces*, vol. 75, 2021, Art. no. 103499.
- [23] Q. Tao, Q. Chen, H. Ding, A. Iftikhar, X. Huang, and X. Cui, "Cross-department secures data sharing in food industry via blockchain-cloud fusion scheme," *Secur. Commun. Netw.*, vol. 2021, pp. 6668339:1–6668339:18, 2021.
- [24] Y. Bao, W. Qiu, and X. Cheng, "Efficient and fine-grained signature for IIoT with resistance to key exposure," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 9189–9205, Jun. 2021.
- [25] Y. Chen, J. Li, C. Liu, J. Han, Y. Zhang, and P. Yi, "Efficient attribute based server-aided verification signature," *IEEE Trans. Serv. Comput.*, vol. 15, no. 6, pp. 3224–3232, Nov./Dec. 2022.
- [26] J. Li, Y. Chen, J. Han, C. Liu, Y. Zhang, and H. Wang, "Decentralized attribute-based server-aid signature in the Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4573–4583, Mar. 2022.
- [27] C. Lin, D. He, X. Huang, K.-K. R. Choo, and A. V. Vasilakos, "BSeIn: A blockchain-based secure mutual authentication with fine-grained access control system for industry 4.0," *J. Netw. Comput. Appl.*, vol. 116, pp. 42–52, 2018.
- [28] P. Datta, T. Okamoto, and K. Takashima, "Efficient attribute-based signatures for unbounded arithmetic branching programs," *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, vol. 104-A, no. 1, pp. 25–57, 2021.
- [29] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. 9th Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2003, pp. 452–473.
- [30] J. Cui, J. Zhang, H. Zhong, R. Shi, and Y. Xu, "An efficient certificateless aggregate signature without pairings for vehicular ad hoc networks," *Inf. Sci.*, vol. 451/452, pp. 1–15, 2018.
- [31] G. Thumbur, G. S. Rao, P. V. Reddy, N. B. Gayathri, D. V. R. K. Reddy, and M. Padmavathamma, "Efficient and secure certificateless aggregate signature-based authentication scheme for vehicular ad hoc networks," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1908–1920, Feb. 2021.
- [32] J. Liu, H. Cao, Q. Li, F. Cai, X. Du, and M. Guizani, "A large-scale concurrent data anonymous batch verification scheme for mobile healthcare crowd sensing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1321–1330, Apr. 2019.
- [33] X. Zhou, D. He, M. K. Khan, W. Wu, and K.-K. R. Choo, "An efficient blockchain-based conditional privacy-preserving authentication protocol for VANETs," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 81–92, Jan. 2023.
- [34] C. P. Schnorr, "Efficient identification and signatures for smart cards," in *Proc. Conf. Theory Appl. Cryptol.*, 1990, pp. 239–252.
- [35] Y. Seurin, "On the exact security of schnorr-type signatures in the random oracle model," in *Proc. 31st Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2012, pp. 554–571.
- [36] N. Kobitz, "A family of jacobians suitable for discrete log cryptosystems," in *Proc. 8th Annu. Int. Cryptol. Conf.*, 1988, pp. 94–99.
- [37] H. P. F. Swinnerton-Dyer and B. J. Birch, "Notes on elliptic curves. I," *J. für die reine und angewandte Mathematik*, vol. 1963, no. 212, pp. 7–25, 1963. [Online]. Available: <http://eudml.org/doc/150565>
- [38] B. J. Birch and H. Swinnerton-Dyer, "Notes on elliptic curves. II," *J. Für Die Rne Und Angewandte Mathematik*, vol. 1963, no. 212, pp. 7–25, 1963.
- [39] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptol.*, vol. 13, no. 3, pp. 361–396, 2000.



Mengjie Zhou received the B.Eng. degree in Internet of Things engineering from Fujian Normal University, Fuzhou, China, in 2021. She is currently working toward the M.S. degree in cyber security with the College of Computer and Cyber Security, Fujian Normal University. Her research interests include blockchain-based authentication technology and its applications.



Wei Wu received the Ph.D. degree from the School of Computer Science and Software Engineering, University of Wollongong, Wollongong, NSW, Australia, in 2011. She is currently a Professor with the College of Mathematics and Informatics, Fujian Normal University, Fuzhou, China. Her research interests include cryptography and information security. She has published more than 20 research papers in refereed international conferences and journals.



Xiaotong Zhou received the bachelor's and master's degrees in information security from Wuhan University, Wuhan, China, in 2012 and 2019 respectively. She is currently working toward the Ph.D. degree with the School of Cyber Science and Engineering, Wuhan University. Her research interests include the area of security and privacy, including privacy protection and blockchain security.



Debiao He (Member, IEEE) received the Ph.D. degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, Wuhan, China in 2009. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He has authored or coauthored more than 100 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and Usenix Security Symposium. His research interests include cryptography and information security, in particular, cryptographic protocols. He was the recipient of the 2018 IEEE Sysems Journal Best Paper Award and the 2019 IET Information Security Best Paper Award. His work has been cited more than 10000 times at Google Scholar. He is in the Editorial Board of several international journals, such as ACM Distributed Ledger Technologies: Research & Practice, Frontiers of Computer Science, and IEEE TRANSACTIONS ON COMPUTERS.